

- Sécurité Web — Black Box Testing
- Quels outils utiliseriez-vous pour du Black Box Testing ?
 - ☒ Réponse attendue
 - ☒ Burp Suite
 - ☒ OWASP ZAP
 - ☒ Nikto
 - Question de réflexion
 - ☒ Réponse attendue
- Comment documenter vos découvertes sans accès au code ?
 - ☒ Réponse attendue
 - Structure recommandée
 - ☒ 1. Contexte du test
 - ☒ 2. Description de la vulnérabilité
 - ☒ 3. Étapes de reproduction (OBLIGATOIRE)
 - ☒ 4. Preuves (Proof of Concept)
 - ☒ 5. Impact métier (TRÈS IMPORTANT)
 - Question de réflexion
 - ☒ Réponse attendue
- **3** Que faire si vous trouvez une vulnérabilité critique ?
 - ☒ Réponse attendue (workflow professionnel)
 - ☒ Étape 1 — Vérifier la vulnérabilité
 - ☒ Étape 2 — Évaluer la gravité
 - ☒ Étape 3 — Documenter proprement
 - ☒ Étape 4 — Notifier de manière responsable
 - ☒ Étape 5 — Proposer des remédiations
 - ☒ Ce qu'il ne faut JAMAIS faire
- Questions bonus
 - ☒ Réponses rapides aux questions bonus
 - ☒ Black Box vs Grey Box : avantages ?
 - ☒ Comment tester la logique métier sans code ?
 - ☒ Pourquoi le rate limiting est crucial ?
 - ☒ Comment détecter une IDOR ?
 - ☒ Quelle est la première chose à faire avant tout test ?

Sécurité Web — Black Box Testing

Quels outils utiliseriez-vous pour du Black Box Testing ?



Réponse attendue

Les principaux outils utilisés en **Black Box Testing web** sont :

◆ Burp Suite

- Proxy interceptant HTTP/HTTPS
- Manipulation des requêtes
- Tests manuels (Repeater, Intruder)
- Scanner automatique (version Pro)

👉 Pourquoi l'utiliser ?

- Analyse fine du trafic
- Tests d'injection
- Fuzzing des paramètres

◆ OWASP ZAP

- Proxy interceptant open source
- Scanner automatisé
- Scripts d'automatisation

👉 Pourquoi l'utiliser ?

- Alternative gratuite à Burp
 - Intégration CI/CD
 - Découverte rapide de vulnérabilités
-

◆ Nikto

- Scanner de serveur web
- Détection de mauvaises configurations
- Recherche de fichiers sensibles

👉 Pourquoi l'utiliser ?

- Phase de reconnaissance
- Audit rapide du serveur

⚠ Limite : pas adapté aux tests logiques avancés.

Question de réflexion

Pourquoi ne faut-il pas se reposer uniquement sur les scans automatiques ?

✅ Réponse attendue

- Présence de faux positifs
 - Présence de faux négatifs
 - Incapacité à tester la logique métier
 - Besoin d'analyse humaine
 - Contexte applicatif non compris par les outils
-

Comment documenter vos découvertes sans accès au code ?

✅ Réponse attendue

Même en **Black Box** , un rapport doit être **clair, reproductible et orienté impact** .

Structure recommandée

◆ 1. Contexte du test

Inclure :

- cible (URL, IP)
 - périmètre
 - date
 - type de test (black box)
-

◆ 2. Description de la vulnérabilité

Exemple :

```
Titre : SQL Injection sur /login
Gravité : Critique
Endpoint : POST /api/auth
Paramètre : username
```

◆ 3. Étapes de reproduction (OBLIGATOIRE)

Le rapport doit permettre à un tiers de reproduire.

Exemple :

1. Intercepter la requête POST /login
2. Modifier `username` en ' `OR 1=1 --`
3. Envoyer la requête
4. Observer la connexion réussie

👉 **Message clé :**

Si ce n'est pas reproductible, la vulnérabilité n'est pas exploitable.

◆ 4. Preuves (Proof of Concept)

Inclure si possible :

- requête HTTP brute
 - réponse serveur
 - captures d'écran
 - payload utilisé
-

◆ 5. Impact métier (TRÈS IMPORTANT)

Exemples :

- accès aux comptes utilisateurs
- fuite de données personnelles
- élévation de privilèges
- compromission du système

Question de réflexion

Pourquoi l'impact métier est-il plus important que le payload ?

✓ Réponse attendue

- Le risque est évalué sur l'impact réel
 - Le payload n'est qu'un moyen technique
 - Le métier décide des priorités de correction
 - Une faille technique sans impact peut être mineure
-

3 Que faire si vous trouvez une vulnérabilité critique ?

Réponse attendue (workflow professionnel)

◆ Étape 1 — Vérifier la vulnérabilité

Toujours confirmer :

- reproductibilité
- absence de faux positif
- périmètre autorisé

⚠ Ne pas sur - exploiter.

◆ Étape 2 — Évaluer la gravité

Méthodes possibles :

- CVSS
 - OWASP Risk Rating
-

◆ Étape 3 — Documenter proprement

Avant toute alerte :

- preuves propres
 - étapes claires
 - impact expliqué
-

◆ Étape 4 — Notifier de manière responsable

Principe : **Responsible Disclosure**

Selon le contexte :

- client
- RSSI
- plateforme bug bounty
- CERT

⚠ Ne jamais divulguer publiquement sans autorisation.

◆ Étape 5 — Proposer des remédiations

Un bon pentester doit :

- expliquer le correctif
 - proposer des bonnes pratiques
 - aider à la priorisation
-

✗ Ce qu'il ne faut JAMAIS faire

- Exploiter en production
 - Exfiltrer des données réelles
 - Modifier des comptes utilisateurs
 - Publier la faille sans délai responsable
-

Questions bonus

Vous pouvez également poser :

- Black Box vs Grey Box : avantages ?
 - Comment tester la logique métier sans code ?
 - Pourquoi le rate limiting est crucial ?
 - Comment détecter une IDOR ?
 - Quelle est la première chose à faire avant tout test ?
-

Réponses rapides aux questions bonus

◆ Black Box vs Grey Box : avantages ?

- **Black Box** : vision réaliste d'un attaquant externe, aucun biais lié au code.
 - **Grey Box** : tests plus ciblés et efficaces grâce à une connaissance partielle du système.
-

◆ Comment tester la logique métier sans code ?

- Observer les flux applicatifs (Burp/ZAP)
 - Tester des enchaînements d'actions
 - Manipuler les paramètres (IDs, rôles, montants)
 - Chercher des incohérences fonctionnelles
-

◆ Pourquoi le rate limiting est crucial ?

- Empêche le brute force
 - Limite les attaques par automatisation
 - Protège les ressources serveur
 - Réduit le risque de déni de service
-

◆ Comment détecter une IDOR ?

- Modifier un identifiant dans une requête
 - Tester l'accès à des ressources d'un autre utilisateur
 - Vérifier l'absence de contrôle d'autorisation côté serveur
-

◆ Quelle est la première chose à faire avant tout test ?

- Obtenir une **autorisation formelle**
- Définir clairement le périmètre
- Vérifier l'environnement de test

Fin du document